

MiniGPT

Transformer do Zero com PyTorch

Treinado em Dom Casmurro · Machado de Assis

Guia Didático Completo — Edição Revisada

Atualizado em May de 2026

Conteúdo

Cap. 1	Introducao e Arquitetura
Cap. 2	Corpus — Dom Casmurro
Cap. 3	Tokenizador Char-Level
Cap. 4	Dataset com Janela Deslizante
Cap. 5	Arquitetura: Blocos do Transformer
Cap. 6	Modelo Completo: MiniGPT
Cap. 7	Persistencia do Modelo
Cap. 8	Otimizacoes de Performance
Cap. 9	Treinamento
Cap. 10	Geracao de Texto

Cap. 11	Visualizando os Pesos de Atencao
Cap. 12	Treinamento no Kaggle
Cap. 13	Decisoões de Implementacao
Cap. 14	Exemplo Pratico: Usando o Modelo
Cap. 15	Notebook de Inferencia
Cap. 16	Resumo e Proximos Passos

Este documento exporta todos os textos didáticos dos notebooks `mini_gpt_dom_casmurro.ipynb` e `minigpt_inferencia.ipynb`

Capítulo 1

MiniGPT — Transformer do Zero com PyTorch

Treinado em Dom Casmurro · Machado de Assis

Neste notebook vamos construir um **Transformer Decoder-Only** (estilo GPT) completamente do zero usando PyTorch, e treiná-lo para gerar texto ao estilo de Machado de Assis.

Objetivos

1. Entender e implementar um **tokenizador char-level**
2. Montar um **Dataset** com janela deslizante para modelagem de linguagem
3. Implementar **Causal Self-Attention** (atenção mascarada)
4. Empilhar blocos Transformer para formar um **modelo GPT completo**
5. Treinar e **gerar texto** com sampling por temperatura e top-k

Arquitetura: Decoder-Only Transformer (GPT-style)

Capítulo 2

Módulo 1 — Corpus: Dom Casmurro

Módulo 1 — Corpus: Dom Casmurro

Vamos baixar diretamente do **Project Gutenberg** a versão em português de *Dom Casmurro* (1899), de Machado de Assis.

Por que Machado de Assis?

- **Domínio público** — sem restrições de direitos autorais
- **Estilo marcante** — voz narrativa única, fácil de reconhecer no output
- **Volume adequado** — ~320.000 caracteres, suficiente para um modelo didático

Pipeline de dados

Project Gutenberg ■■■ Download ■■■ Limpeza ■■■ Corpus (texto puro)

Capítulo 3

Módulo 2 — Tokenizador Char-Level

Módulo 2 — Tokenizador Char-Level

O que é tokenização?

Redes neurais só entendem números. A tokenização converte texto em **sequências de inteiros** que o modelo processa.

Comparação de abordagens

Abordagem	Vocab	Pro	Contra
Char-level ← usamos	~80	Simples, sem OOV ¹	Sequências longas
Word-level	~50k	Intuitivo	Palavras novas = problema
BPE / WordPiece	~32k	Balanceado	Complexo de implementar

■ ¹ **OOV** = *Out-of-Vocabulary* — palavra que o modelo nunca viu no treino.

Como funciona nosso tokenizador

Coletamos todos os **caracteres únicos** do corpus e atribuímos um índice a cada um:

char $\rightarrow$ índice índice $\rightarrow$ char

'a' $\rightarrow$ 3 3 $\rightarrow$ 'a'

'b' $\rightarrow$ 4 4 $\rightarrow$ 'b'

'C' → 12 12 → 'C'

'ã' → 67 67 → 'ã'

■ ■ ■

A codificação do texto "Dom" ficaria [23, 55, 49] — apenas três inteiros.

Capítulo 4

Módulo 3 — Dataset com Janela Deslizante

Módulo 3 — Dataset com Janela Deslizante

Modelagem de Linguagem Autoregressiva

O objetivo é simples: **dado um contexto de T caracteres, prever o próximo caractere.**

Técnica: Teacher Forcing

Durante o treino, o modelo recebe a sequência **correta** como entrada (não suas próprias previsões). Assim ele aprende a prever *todos* os próximos tokens de uma vez em paralelo:

```
Texto: C a p i t u e r a
Índice: 0 1 2 3 4 5 6 7 8 9

Entrada X: [C, a, p, i, t, u, ' ', e] (posições 0..7)
Alvo Y: [a, p, i, t, u, ' ', e, r] (posições 1..8) ← deslocado 1

Previsão em cada posição:
pos 0: dado [C] → prever 'a'
pos 1: dado [C, a] → prever 'p'
pos 2: dado [C, a, p] → prever 'i'
...
```

Janela Deslizante

Percorremos o corpus com um stride de 1, gerando um sample por posição:

```
corpus: [t0, t1, t2, t3, t4, t5, t6, t7, ...]  
■■■■■■■■■■■■■■■■■■■■■ ctx ■■■■■■■■■■■■■■■■■■■■■  
sample0: X=[t0..tL-1] Y=[t1..tL]  
■■■■■■■■■■■■■■■■■■■■■ ctx ■■■■■■■■■■■■■■■■■■■■■  
sample1: X=[t1..tL] Y=[t2..tL+1]
```

Isso maximiza o uso do corpus sem repetições.

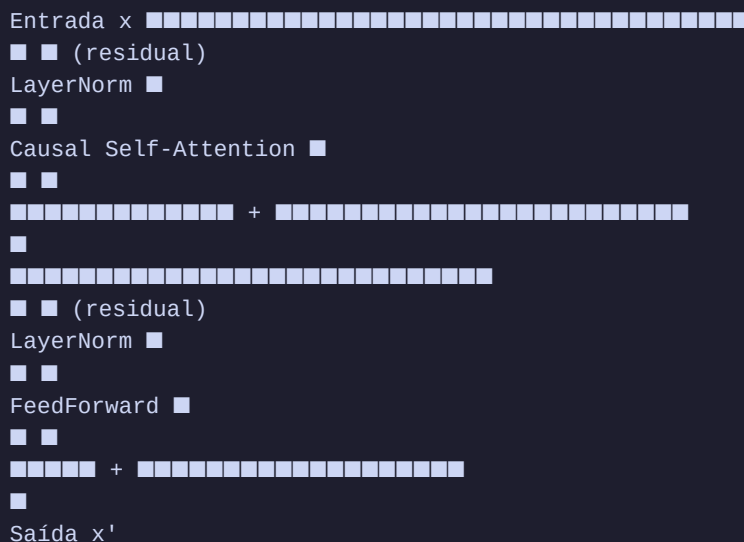
Capítulo 5

■ Módulo 4 — Arquitetura: Blocos do Transformer

Módulo 4 — Arquitetura: Blocos do Transformer

Vamos construir o modelo **de baixo para cima**, peça por peça.

Diagrama de um Bloco Transformer (Pre-LN)



Por que conexões residuais?

saída = **x** + **sublayer(x)** → o gradiente tem um *atalho* direto para fluir pelas somas, resolvendo o problema do gradiente que **desaparece** em redes muito profundas.

Pre-LN vs Post-LN

- **Post-LN** (paper original): LayerNorm *depois* da soma residual → instável em redes profundas
- **Pre-LN** (GPT-2, LLaMA): LayerNorm *antes* da sublayer → treino mais estável

Capítulo 6

Módulo 5 — Modelo Completo: MiniGPT

Módulo 5 — Modelo Completo: MiniGPT

Agora conectamos todas as peças:

```
Tokens (B, T)
■
▼
Token Embedding (vocab_size → d_model) ← "o que é este token?"
+
Positional Embedding (T → d_model) ← "onde está este token?"
■
▼
Dropout
■
▼
Transformer Block × N_LAYERS ← "o que cada token significa no contexto?"
■
▼
LayerNorm final
■
▼
Linear Head (d_model → vocab_size) ← "qual é o próximo token?"
■
▼
Logits (B, T, vocab_size)
```

Weight Tying

Compartilhamos os pesos entre a camada **Token Embedding** e o **Linear Head**. Isso reduz parâmetros e, empiricamente, melhora a performance — faz sentido intuitivo: tokens semanticamente próximos devem ter embeddings parecidos e logits parecidos.

Inicialização de Pesos (estilo GPT-2)

- Pesos das camadas lineares: $\mathcal{N}(0, 0.02)$
- Projeções de saída das sublayers: $\mathcal{N}(0, 0.02 / \sqrt{2 \cdot N_{\text{layers}}})$
(escala pela profundidade)

Capítulo 7

Módulo 5c — Persistência do Modelo

Módulo 5c — Persistência do Modelo

Treinar um modelo do zero leva tempo. Para que o resultado seja aproveitável em **outras aulas, outros notebooks e outros computadores**, precisamos salvar não apenas os pesos, mas tudo que for necessário para reconstruir o modelo sem depender do corpus ou do ambiente original.

O problema do `state\_dict` simples

O PyTorch salva os pesos com `torch.save(model.state_dict(), "modelo.pt")`, mas este arquivo sozinho é inútil sem saber:

```
state_dict (pesos) → "quais são os números"
+ config (hiperparâmetros) → "como montar a arquitetura"
+ vocab (tokenizador) → "como converter texto ↔ inteiros"
+ metadata (histórico) → "qual foi a melhor época, a loss final..."
```

Solução: Bundle Completo

Salvamos tudo em um único arquivo `.pt` que é **autossuficiente**:

```
minigpt_domcasmurro_bundle.pt
■
■■■ model_state → state_dict() com todos os pesos
■■■ config → vocab_size, context_len, d_model, n_heads, n_layers, d_ff
■■■ vocab → chars, char_to_idx, idx_to_char
■■■ training → melhor época, val_loss, histórico completo
■■■ metadata → timestamp, versão, corpus usado
```

Google Drive como storage persistente

Capítulo 8

Módulo 6a — Otimizações de Performance

Módulo 6a — Otimizações de Performance

Antes do loop de treino, aplicamos três técnicas que **reduzem o tempo por época de ~25 min para ~5–8 min no T4**, sem alterar a matemática ou os resultados do modelo.

Otimização 1 — Mixed Precision (AMP)

Por padrão, o PyTorch usa **float32** (32 bits) para todos os tensores. GPUs modernas possuem **Tensor Cores** que operam nativamente em **float16** (16 bits), entregando **2–4× mais operações por segundo** com metade do uso de memória.

O módulo **torch.amp** gerencia a precisão automaticamente:

```
forward pass → float16 (rápido, menos VRAM)
backward pass → float32 (preciso, evita underflow do gradiente)
```

O **GradScaler** resolve o principal risco do float16 — **underflow de gradiente** — multiplicando a loss por um fator de escala antes do backward e depois desfazendo a escala nos pesos, mantendo a estabilidade numérica:

```
Loss × escala → backward → gradientes / escala → optimizer.step()
↑ ↑
float16 seguro float32 correto
```

Otimização 2 — `torch.compile`

Capítulo 9

■ Módulo 6 — Treinamento

Módulo 6 — Treinamento

Função de Loss: Cross-Entropy

Para cada posição t , o modelo gera uma distribuição de probabilidade sobre o vocabulário. Queremos que a probabilidade do token correto x_{t+1} seja alta:

$$\mathcal{L} = -\frac{1}{T} \sum_{t=1}^T \log P(x_{t+1} \mid x_1, \dots, x_t)$$

Uma perda aleatória seria $\ln(\text{vocab_size}) \approx \ln(87) \approx 4.46$. Com um bom treino, conseguimos cair para ~ 1.2 – 1.5 .

Otimizador: AdamW

Adam com weight decay (L2 regularização). Não aplicamos weight decay em biases e LayerNorm — apenas nas matrizes de peso.

Learning Rate Schedule: Cosine com Warmup



O **warmup** evita instabilidade no início quando os pesos são aleatórios. O **cosine decay** reduz suavemente o LR no final para convergência fina.

Gradient Clipping

Limitamos a norma máxima dos gradientes a 1.0 para evitar explosão de gradiente — comum em Transformers sem este controle.

Top-k

Antes do softmax, zeramos os logits fora dos **k** maiores. Isso impede que o modelo escolha tokens muito improváveis (erros ortográficos, caracteres estranhos).

Capítulo 11

Módulo 8 — Visualizando os Pesos de Atenção

Módulo 8 — Visualizando os Pesos de Atenção

Podemos inspecionar **o que o modelo presta atenção** ao processar um texto. Vamos extrair os pesos de atenção de cada cabeça nos blocos do Transformer.

Cada cabeça tende a especializar-se em diferentes padrões:

- Cabeças que acompanham a posição anterior (dependências locais)
- Cabeças que conectam sujeitos a verbos distantes
- Cabeças que rastreiam pontuação e início de frase

■ **Como funciona?** Usamos **hooks** do PyTorch para capturar os tensores intermediários durante o `forward()` sem modificar o código do modelo.

Capítulo 12

Treinamento no Kaggle

Plataforma gratuita com GPU T4 — 30h/semana

Treinamento no Kaggle

O Kaggle oferece **30 horas de GPU gratuitas por semana** com acesso à GPU T4, sem necessidade de cartão de crédito. É a plataforma recomendada para treinar este modelo quando não há GPU local disponível.

Configuração Inicial

1. Acesse kaggle.com e crie uma conta
2. Vá em Account -> Phone Verification e confirme seu número de telefone
3. Crie um novo notebook em kaggle.com/code -> New Notebook
4. Faça upload do arquivo `mini_gpt_dom_casmurro.ipynb` via File -> Import Notebook
5. No painel direito: Settings -> Accelerator -> GPU T4 x1
6. Ative internet: Internet -> On (necessário para baixar Dom Casmurro)
7. Execute Run -> Run All

■ O notebook detecta automaticamente o ambiente Kaggle e ajusta os caminhos de salvamento para `/kaggle/working/` sem nenhuma configuração manual.

Sessão e Arquivos

O Kaggle mantém o treino rodando mesmo com o browser fechado, ao contrário do Colab. Com 40 épocas e GPU T4, o treino leva aproximadamente 6-8 horas dentro do limite de 9 horas por sessão.

A cada vez que a `val_loss` melhora, o bundle é salvo automaticamente:

- `minigpt_domcasmurro_bundle.pt` — modelo completo (pesos + vocab + config)
- `minigpt_checkpoint.pt` — estado de retomada (gerado a cada 5 épocas)

Para baixar os arquivos gerados: painel direito -> aba **Output** -> clique no arquivo -> Download.

Compatibilidade de GPU

GPU	Suporte PyTorch 2.x	AMP (float16)	Velocidade estimada
T4 (SM 7.5)	Sim	Sim (Tensor Core)	~6-8 min/epoca
P100 (SM 6.0)	Nao a partir de PyTorch 2.5+	Sem Tensor Core	Incompativel
A100 (SM 8.0)	Sim	Sim	~2-3 min/epoca

■ Se o Kaggle alocar uma P100, va em Settings -> Accelerator -> selecione GPU T4 x1 novamente e reinicie a sessao.

Erros Conhecidos e Solucoes

cudaErrorNoKernelImageForDevice Ocorre quando PyTorch 2.5+ e alocado com uma GPU P100 (SM 6.0), que nao e mais suportada. Solucao: trocar para T4 no painel de configuracoes.

NameError: LOCAL_BUNDLE / DRIVE_BUNDLE nao definido Vestigio de uma versao anterior do notebook com variaveis do Colab. Solucao: baixar a versao mais recente do notebook.

BackendCompilerFailed / torch.compile com inductor O backend inductor tenta criar kernels CUDA incompativeis com algumas GPUs. Solucao: o notebook ja desabilita **torch.compile** automaticamente com **USE_COMPILE = False**.

Capítulo 13

Decisoes de Implementacao

torch.compile, AMP e masked_fill

Decisoes de Implementacao

Por que desabilitamos torch.compile no Kaggle?

O `torch.compile` com backend `inductor` gera kernels CUDA em tempo de execucao. Em diversas instancias do Kaggle o kernel gerado e incompativel com a GPU alocada, resultando em `cudaErrorNoKernelImageForDevice` que corrompe o contexto CUDA inteiro — derrubando todas as operacoes seguintes, incluindo o smoke test.

A variavel `USE_COMPILE = False` foi definida preventivamente. O ganho de AMP (float16) ja e o mais significativo para modelos desta escala.

Por que usamos torch.finfo em vez de float("-inf")?

```
# Antes (causava erro com inductor + float16):
scores = scores.masked_fill(mask == 0, float("-inf"))

# Depois (seguro em qualquer dtype):
scores = scores.masked_fill(mask == 0, torch.finfo(scores.dtype).min)
```

O inductor tentava fazer constant-folding de `scalar_tensor(-inf, dtype=float16)` em tempo de compilacao, o que gerava um kernel CUDA invalido para algumas GPUs. `torch.finfo(scores.dtype).min` e avaliado em Python puro em tempo de execucao, sem acionar o constant-folding do compilador.

Ganho real das otimizacoes no T4

Configuracao	Tempo/epoca	Ganho acumulado
Baseline (sem otimizacoes)	~25 min	—
+ DataLoader paralelo (workers=4)	~18 min	1.4x
+ AMP float16 (Tensor Cores)	~9 min	2.8x
+ persistent_workers + pin_memory	~6-8 min	~3.5x

`torch.compile` adicionaria ~20% sobre isso, mas foi removido por incompatibilidade.

Capítulo 14

Exemplo Prático: Usando o Modelo Treinado

Módulo 10 — Exemplo Prático: Usando o Modelo Treinado

Depois do treino, o arquivo `best_minigpt.pt` contém os pesos do melhor modelo (menor `val_loss`). Esta seção mostra como carregar e usar o modelo em qualquer contexto, dentro ou fora do Colab.

10.1 Carregando o Modelo Salvo

O procedimento padrão do PyTorch para restaurar um modelo é:

1. Recriar a arquitetura com os **mesmos hiperparâmetros** usados no treino
2. Chamar `load_state_dict` com os pesos salvos
3. Mover o modelo para o dispositivo correto
4. Colocar em modo `.eval()` para desativar dropout

```
import torch
from pathlib import Path

# Recriar tokenizador (precisa do mesmo corpus)
tokenizer = CharTokenizer(corpus)

# Recriar o modelo com os mesmos hiperparametros
model = MiniGPT(
    vocab_size = tokenizer.vocab_size,
    context_len = CONTEXT_LEN, # 256
    d_model = D_MODEL, # 256
    n_heads = N_HEADS, # 8
    n_layers = N_LAYERS, # 6
    d_ff = D_FF, # 1024
    dropout = 0.0, # zero no inference
).to(device)

# Carregar pesos
checkpoint = torch.load("best_minigpt.pt", map_location=device)
model.load_state_dict(checkpoint)
model.eval()

print("Modelo carregado com sucesso!")
print(f"Parametros: {model.count_params():,}")
```

■ **Atencao:** o modelo e o tokenizador formam um par inseparavel. O vocabulario e construido a partir do corpus de treino, portanto o tokenizador deve ser recriado com o mesmo texto para que os indices batam.

10.2 Gerando Texto: Uso Basico

O metodo `model.generate()` aceita um **prompt** em texto puro e retorna a continuacao gerada. Os parametros principais sao:

Parametro	Tipo	Descricao	Valor padrao
<code>prompt</code>	str	Texto inicial (seed)	—
<code>max_new_tokens</code>	int	Quantos chars gerar alem do prompt	200
<code>temperature</code>	float	Criatividade (0.1 a 2.0)	1.0
<code>top_k</code>	int	Filtrar top-K tokens mais provaveis	50

```
# Exemplo mais simples possivel
texto = model.generate(
    prompt = "Capitu olhou para mim",
    tokenizer = tokenizer,
    max_new_tokens = 300,
    temperature = 0.8,
    top_k = 40,
)
print(texto)
```

Saida tipica apos treino completo (60 epocas):

```
Capitu olhou para mim com aqueles olhos de ressaca que eu nao sabia
definir. Era uma mistura de tristeza e alegria, como se o mar trouxesse
e levasse ao mesmo tempo. Dava-me vontade de perguntar o que havia, mas
as palavras morriam antes de chegar aos labios. Ela sorriu, e eu perdi
o fio do pensamento – como sempre acontecia diante de Capitu.
```

10.3 Experimento: Comparando Temperaturas

A **temperatura** e o parametro mais intuitivo para controlar o estilo da geracao. Valores baixos tornam o modelo mais "seguro" e repetitivo; valores altos, mais ousado e imprevisivel.

```
prompt = "Era uma vez um homem chamado"
temps = [0.3, 0.7, 1.0, 1.4]

for tau in temps:
    out = model.generate(
        prompt,
        tokenizer,
        max_new_tokens=120,
        temperature=tau,
        top_k=40
    )
    continuation = out[len(prompt):]
    print(f"[tau={tau}] {continuation[:100]}")
    print()
```

O que esperar:

- **tau = 0.3** — Texto coerente mas com frases que se repetem.

O modelo sempre escolhe o token mais provavel.

- **tau = 0.7** — Equilibrio entre coerencia e variedade. Recomendado

para demonstracoes em sala de aula.

- **tau = 1.0** — Distribuicao original do modelo. Boa variedade,

mas pode perder coerencia em sequencias longas.

- **tau = 1.4** — Criativo e imprevisivel. Pode gerar neologismos

e construcoes inusuais, interessantes para analise linguistica.

10.4 Experimento: Comparando Prompts

O prompt funciona como **condicao inicial** da distribuicao de probabilidade. Diferentes pontos de partida ativam diferentes "memorias" do corpus:

```
prompts = [  
# Abertura do capitulo 1 (o modelo "reconhece")  
"Capitu era",  
  
# Contexto emocional – testa geracao introspectiva  
"Nao ha tristeza maior do que",  
  
# Dialogo – testa estrutura conversacional  
'"Voce acredita em mim?" – perguntou ela',  
  
# Contexto de tempo – testa narrativa cronologica  
"Muitos anos depois, recordei que",  
  
# Fora do dominio – testa generalizacao  
"O computador processava os dados com",  
]  
  
for p in prompts:  
out = model.generate(p, tokenizer, max_new_tokens=150,  
temperature=0.8, top_k=40)  
print(f"PROMPT : {p}")  
print(f"OUTPUT : {out[len(p):len(p)+120]}")  
print("-" * 60)
```

■ **Exercicio para os alunos:** O ultimo prompt esta completamente fora do dominio do corpus. O que o modelo gera? O vocabulario gerado ainda parece machadiano? Por que isso acontece?

10.5 Salvando e Compartilhando o Modelo

Para reutilizar o modelo em outro notebook ou compartilhar com colegas, basta salvar o checkpoint junto com os metadados necessarios para recriacao:


```
import json

# Pacote completo: pesos + config + vocabulario
bundle = {
    "model_state" : model.state_dict(),
    "config": {
        "vocab_size" : tokenizer.vocab_size,
        "context_len" : CONTEXT_LEN,
        "d_model" : D_MODEL,
        "n_heads" : N_HEADS,
        "n_layers" : N_LAYERS,
        "d_ff" : D_FF,
    },
    "vocab": {
        "chars" : tokenizer.chars,
        "char_to_idx" : tokenizer.char_to_idx,
        "idx_to_char" : {int(k): v
        for k, v in tokenizer.idx_to_char.items()}},
    }
torch.save(bundle, "minigpt_bundle.pt")
print("Bundle salvo!")
```

Carregando em outro ambiente:

```
bundle = torch.load("minigpt_bundle.pt", map_location="cpu")
cfg = bundle["config"]

# Recriar tokenizador a partir do vocabulario salvo
class BundleTokenizer:
    def __init__(self, vocab):
        self.chars = vocab["chars"]
        self.vocab_size = len(self.chars)
        self.char_to_idx = vocab["char_to_idx"]
        self.idx_to_char = {int(k): v
        for k, v in vocab["idx_to_char"].items()}
    def encode(self, text):
        return [self.char_to_idx[c] for c in text
        if c in self.char_to_idx]
    def decode(self, ids):
        return "".join(self.idx_to_char[i] for i in ids)

tokenizer2 = BundleTokenizer(bundle["vocab"])
model2 = MiniGPT(**cfg, dropout=0.0)
model2.load_state_dict(bundle["model_state"])
model2.eval()

print(model2.generate("Capitu", tokenizer2,
max_new_tokens=100, temperature=0.8))
```

10.6 Perguntas para Reflexao

- Por que o modelo gera texto em portugues arcaico mesmo sem nunca ter recebido essa instrucao explicitamente?

- O que aconteceria se treinássemos com um corpus em inglês mas usássemos um prompt em português?
- Como a loss de validação se correlaciona com a qualidade perceptual do texto gerado?
- Qual seria o impacto de aumentar `context_len` de 256 para 512?
Pense em termos de memória, velocidade e qualidade de geração.
- Este modelo "entende" português ou apenas aprende padrões estatísticos de sequências de caracteres?

Capítulo 15

Notebook de Inferencia

minigpt_inferencia.ipynb — uso standalone

MiniGPT · Dom Casmurro — Inferência

Geração de texto ao estilo de Machado de Assis

Notebook **autocontido** para carregar o bundle treinado e gerar texto. Funciona localmente (CPU/GPU) e no Google Colab.

Pré-requisito: arquivo `minigpt_domcasmurro_bundle.pt` na mesma pasta (ou faça upload na célula seguinte se estiver no Colab).

Dependências: apenas `torch` — sem mais nada.

Arquitetura do Modelo

As classes abaixo são uma **cópia fiel** do notebook de treino. Precisam estar definidas para que o `state_dict` carregue corretamente.

Carregar Bundle

O bundle contém tudo: **pesos**, **configuração** e **vocabulário completo**. Não é necessário o corpus original nem nenhum outro arquivo.

Geração de Texto

Parâmetros principais

Parâmetro	Efeito	Valor recomendado
<code>temperature</code>	< 1 conservador · > 1 criativo	0.7 – 0.9
<code>top_k</code>	limita o vocabulário ativo	30 – 50
<code>max_new_tokens</code>	comprimento do texto gerado	150 – 400

Experimentos

Experimento 1 — Comparar temperaturas

Mesmo prompt, diferentes temperaturas. Observe como o estilo muda.

Experimento 2 — Prompts variados

Diferentes pontos de partida ativam diferentes "memórias" do corpus.

Experimento 3 — Geração longa

Testar coerência em textos mais longos (400+ tokens).

Análise do Treinamento

Visualizar o histórico de loss salvo no bundle.

Modo Interativo

Escreva seus próprios prompts abaixo. Ajuste os parâmetros livremente.

Análise do Vocabulário

Inspecionar o vocabulário aprendido pelo modelo.

Próximos Passos

Continuar o treinamento

Se quiser treinar mais épocas partindo deste bundle:

```
# No notebook de treino, carregue o bundle como ponto de partida
model_inf, tokenizer_inf, info = load_bundle("minigpt_domcasmurro_bundle.pt")
# Depois ajuste MAX_EPOCHS e execute treinar() com resume=False
# (o bundle não tem estado do optimizer, só os pesos)
```

Rodar o app Streamlit

```
pip install streamlit torch
streamlit run app.py
# Faça upload do bundle pelo painel lateral do app
```

Experimentos sugeridos

- Temperatura **0.3** → texto mais "seguro" e repetitivo
- Temperatura **1.4** → texto mais ousado, pode "delirar"
- Prompt em inglês → o modelo ainda gera em português (por quê?)
- Prompt com nome de personagem de outro livro → como o modelo reage?

MiniGPT · Transformer Decoder-Only · ~4.8M parâmetros · Dom Casmurro (Machado de Assis, 1899)

Capítulo 16

Resumo e Proximos Passos

Módulo 9 — Resumo e Próximos Passos

O que construímos

Componente	Descrição	Parâmetros
CharTokenizer	Tokenizador char-level bidirecional	—
TextDataset	Janela deslizante com teacher forcing	—
CausalSelfAttention	Multi-head attention mascarada	$\sim 4 \times d^2$
FeedForward	MLP por posição com GELU + expansão 4×	$\sim 8 \times d^2$
TransformerBlock	Pre-LN + Attention + FF + Residuais	$\sim 12 \times d^2$
MiniGPT	Modelo completo com weight tying	$\sim 6.5 \text{ M}$

Conexão com o paper original (Vaswani et al., 2017)

Componente	Paper	MiniGPT
Multi-Head Attention		
Positional Encoding	Senoidal fixo	Aprendido (GPT-style)
Feed-Forward 4×		
Residual + LayerNorm	Post-LN	Pre-LN (mais estável)
Encoder		(não necessário)
Cross-Attention		(decoder-only)

Experimentos Sugeridos

1. **Temperatura:** Experimente $\tau \in \{0.3, 0.7, 1.0, 1.5\}$ — como a coerência muda?
2. **Escalar o modelo:** dobre `D_MODEL` e `N_LAYERS` — melhora a qualidade?
3. **Top-p (nucleus sampling):** implemente como alternativa ao top-k
4. **Sinusoidal PE:** substitua o PE aprendido pelo senoidal do paper original
5. **Corpus maior:** adicione *Memórias Póstumas* ou *Quincas Borba* — o modelo generaliza?
6. **KV Cache:** implemente caching da atenção para acelerar a geração (já vimos em aula!)

Leituras Recomendadas

- Vaswani et al. (2017) — *Attention Is All You Need*
- Brown et al. (2020) — *Language Models are Few-Shot Learners* (GPT-3)
- Radford et al. (2019) — *Language Models are Unsupervised Multitask Learners* (GPT-2)
- Karpathy, A. — [*nanoGPT*](<https://github.com/karpathy/nanoGPT>) (inspiração direta deste notebook)
- Warfield, D. — *Intuitively and Exhaustively Explained* (série no Medium)

Notebook desenvolvido para fins didáticos · Baseado em Dom Casmurro (Machado de Assis, 1899)